

Visual-Inertial Odometry

이론과 구현 — 학습 안내서

Intel RealSense T265 · Extended Kalman Filter · Stereo VO

- Ch 1** VIO 개요 — 왜 필요한가?
- Ch 2** 3D 공간과 쿼터니언
- Ch 3** IMU 센서 모델
- Ch 4** Extended Kalman Filter 이론
- Ch 5** IMU 예측 단계 — F 행렬과 Q 행렬
- Ch 6** 중력 정렬 초기화
- Ch 7** 피시아이 카메라 — Kannala-Brandt
- Ch 8** 스테레오 비주얼 오도메트리
- Ch 9** EKF 측정 업데이트와 게이팅
- Ch 10** ZUPT — 정지 검출
- Ch 11** 전체 시스템 파이프라인

Chapter 1 — VIO란 무엇인가?

1.1 자기 위치 추정 (Ego-motion Estimation)

로봇이나 드론이 자신의 위치와 자세를 실시간으로 추정하는 문제를 자기 위치 추정 (Ego-motion Estimation) 이라고 합니다. GPS가 없는 실내 환경이나 빠른 움직임이 필요한 상황에서는 카메라와 IMU(관성 측정 장치)를 결합한 VIO(Visual-Inertial Odometry)를 사용합니다.

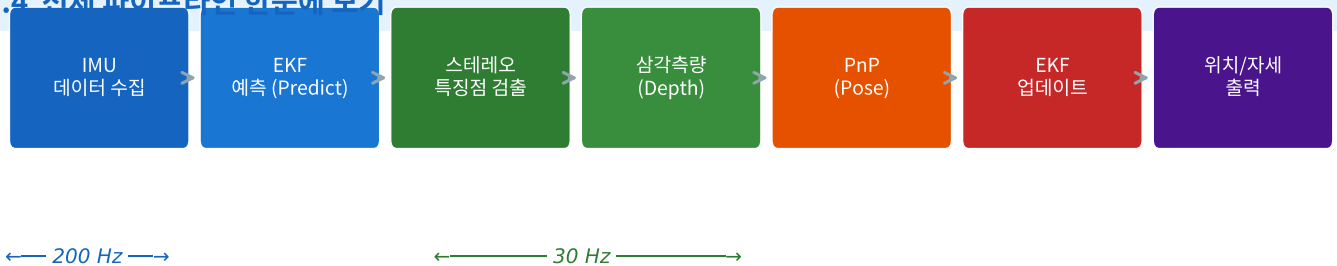
1.2 센서별 특성과 상보적 관계

IMU (가속도계 + 자이로)	카메라 (스테레오)	VIO (IMU + 카메라)
[+] 200 Hz 고주파 측정 [+] 빠른 움직임에 강함 [-] 이중 적분 시 오차 누적 (드리프트) [-] 스케일 절대값 불명확	[+] 30 Hz, 풍부한 텍스처 정보 [+] 스테레오 베이스라인으로 metric 스케일 [-] 빠른 움직임에 모션 블러 [-] 어두운 환경에 취약	[+] IMU로 고주파 예측 (Predict) [+] 카메라로 오차 보정 (Update) [+] Metric 스케일 + 낮은 드리프트 <-- 두 센서의 장점만 취함

1.3 Intel RealSense T265 하드웨어

카메라: 피시아이 스테레오 (FOV ~160°), 848×800, 30 fps
IMU: 가속도계 62 Hz, 자이로스코프 200 Hz, factory calibrated
베이스라인: 6.4 cm (스테레오 baseline) — metric 깊이 복원의 기준
렌즈 모델: Kannala-Brandt4 (피시아이 왜곡 4계수)

1.4 전체 파이프라인의 하늘에 보기



핵심 개념: IMU는 빠르지만 드리프트, 카메라는 느리지만 정확합니다.
 EKF가 두 센서를 통계적으로 최적 융합하여 고주파 + 낮은 드리프트를 동시에 달성합니다.

Chapter 2 — 3D 공간과 쿼터니언

2.1 좌표계 (Reference Frames)

VIO에서는 세 가지 좌표계가 사용됩니다:

- 세계 좌표계 (World frame, W): 초기화 시점에 고정된 관성 기준계. Z축이 중력 반대 방향 (+위쪽).
- 몸체 좌표계 (Body frame, B): IMU가 붙어 있는 디바이스. 디바이스와 함께 회전/이동.
- 카메라 좌표계 (Camera frame, C): 왼쪽 카메라의 광학 중심. Z축이 카메라가 바라보는 방향.

2.2 회전 행렬 (Rotation Matrix)

3×3 직교 행렬 R로 회전을 표현합니다. R의 열벡터는 회전 후 좌표계의 축 방향입니다.

$$\mathbf{R} \in SO(3), \quad \mathbf{R}^T \mathbf{R} = \mathbf{I}, \quad \det(\mathbf{R}) = 1$$

성질

$$\mathbf{v}_W = \mathbf{R}_{BW} \mathbf{v}_B$$

(몸체 → 세계 변환)

2.3 쿼터니언 (Quaternion) — 왜 쓰는가?

회전 행렬은 9개 숫자지만 자유도는 3입니다. 쿼터니언은 4개 숫자로 모든 회전을 표현하며

짐벌 락(Gimbal Lock)이 없고 수치 오차가 누적되어도 재정규화가 쉽습니다.

$$\mathbf{q} = [w, x, y, z]^T, \quad w^2 + x^2 + y^2 + z^2 = 1$$

정의

$$\hat{n} \ \theta \Rightarrow \mathbf{q} = [\cos \frac{\theta}{2}, \hat{n} \sin \frac{\theta}{2}]$$

해석: 축 으로 각도 회전

>> 쿼터니언 곱셈 (Hamilton Product) — 두 회전의 합성

$$w = w_1 w_2 - x_1 x_2 - y_1 y_2 - z_1 z_2$$

Hamilton Product

$$x = w_1 x_2 + x_1 w_2 + y_1 z_2 - z_1 y_2$$

$$y = w_1 y_2 - x_1 z_2 + y_1 w_2 + z_1 x_2$$

$$z = w_1 z_2 + x_1 y_2 - y_1 x_2 + z_1 w_2$$

>> 쿼터니언 → 회전 행렬

$$R_{00} = 1 - 2(y^2 + z^2), \quad R_{01} = 2(xy - wz), \quad R_{02} = 2(xz + wy)$$

R(q)

$$R_{10} = 2(xy + wz), \quad R_{11} = 1 - 2(x^2 + z^2), \quad R_{12} = 2(yz - wx)$$

$$R_{20} = 2(xz - wy), \quad R_{21} = 2(yz + wx), \quad R_{22} = 1 - 2(x^2 + y^2)$$

직관: $\mathbf{q} = [1, 0, 0, 0]$ 은 단위 회전(항등)입니다. $\mathbf{q}_w = \cos(\theta/2)$ 이므로 작은 각도에서 $w \approx 1$.

정규화: 통합 오차로 $\|\mathbf{q}\| \neq 1$ 이 되면 $\mathbf{q} / \|\mathbf{q}\|$ 로 재정규화합니다 (매 EKF 스텝마다 수행).

Chapter 3 — IMU 센서 모델

3.1 가속도계 모델

가속도계는 비력(Specific Force) = 실제 가속도 - 중력 을 몸체 좌표계에서 측정합니다.

측정값에는 바이어스와 가우시안 노이즈가 섞입니다:

$$\tilde{\mathbf{a}} = \mathbf{R}^T(\mathbf{a}_W - \mathbf{g}) + \mathbf{b}_a + \mathbf{n}_a$$

가속도계

$$\mathbf{n}_a \sim \mathcal{N}(\mathbf{0}, \sigma_a^2 \mathbf{I}), \quad \dot{\mathbf{b}}_a \sim \mathcal{N}(\mathbf{0}, \sigma_{ba}^2 \mathbf{I})$$

여기서 R은 세계→몸체 회전, $\mathbf{g} = [0, 0, -9.81] \text{ m/s}^2$ (세계 좌표 중력), $\mathbf{n}_a$ 는 백색 노이즈,

$\mathbf{b}_a$ 는 천천히 변하는 바이어스(랜덤 워크)입니다. T265 스펙: $\sigma_a \approx 0.01 \text{ m/s}^2/\sqrt{\text{Hz}}$.

3.2 자이로스코프 모델

자이로는 각속도 ω 를 몸체 좌표계에서 측정합니다 (단위: rad/s):

$$\tilde{\omega} = \omega + \mathbf{b}_g + \mathbf{n}_g$$

자이로스코프

$$\mathbf{n}_g \sim \mathcal{N}(\mathbf{0}, \sigma_g^2 \mathbf{I}), \quad \dot{\mathbf{b}}_g \sim \mathcal{N}(\mathbf{0}, \sigma_{bg}^2 \mathbf{I})$$

T265 스펙: $\sigma_g \approx 0.002 \text{ rad/s}/\sqrt{\text{Hz}}$. 자이로 바이어스는 정지 상태에서 측정값 평균으로

초기 추정합니다 (Ch 6 참조).

3.3 바이어스의 의미와 중요성

왜 바이어스 추정이 중요한가?

- 가속도계 바이어스 $\mathbf{b}_a$ 가 0.01 m/s^2 오차가 있으면:
속도 오차 = $0.01 \times t \rightarrow 10\text{초 후 } 0.1 \text{ m/s}$ 속도 오차
위치 오차 = $0.5 \times 0.01 \times t^2 \rightarrow 10\text{초 후 } 0.5 \text{ m}$ 위치 오차
- 자이로 바이어스 $\mathbf{b}_g$ 가 0.01 rad/s 오차가 있으면:
각도 오차 = $0.01 \times t \rightarrow 10\text{초 후 } 0.1 \text{ rad} \approx 5.7^\circ$ 자세 오차

3.4 IMU 전처리: 바이어스 보정과 세계 좌표 변환

EKF 예측 단계에서 IMU 측정값을 사용하기 전에 추정된 바이어스를 제거합니다:

$$\mathbf{a}_{body} = \tilde{\mathbf{a}} - \hat{\mathbf{b}}_a$$

IMU 전처리
(바이어스 보정 가속도)

$$\omega_{body} = \tilde{\omega} - \hat{\mathbf{b}}_g$$

(바이어스 보정 각속도)

$$\mathbf{a}_W = \mathbf{R} \mathbf{a}_{body} + \mathbf{g}$$

(세계 좌표 가속도)

직관: IMU는 항상 몸체 좌표계에서 값을 줍니다. EKF의 상태는 세계 좌표계 기준이므로

회전 행렬 R로 변환해야 합니다. 이 변환이 틀리면 중력 방향이 뒤섞여 발산합니다.

Chapter 4 — Extended Kalman Filter

4.1 칼만 필터의 직관 — 두 정보의 최적 융합

당신이 위치를 두 가지 방법으로 추정한다고 가정합니다:

- ① 지도로 예측: "10m 앞에 있을 것" (불확실도 ± 2 m)
- ② GPS로 측정: "현재 위치는 11m" (불확실도 ± 1 m)

최적 답은 두 정보를 불확실도에 반비례하여 가중 평균한 것입니다:

$$\hat{x} = \frac{\sigma_2^2}{\sigma_1^2 + \sigma_2^2} x_1 + \frac{\sigma_1^2}{\sigma_1^2 + \sigma_2^2} x_2$$

1D 직관

$$K = \frac{\sigma_1^2}{\sigma_1^2 + \sigma_2^2}$$

칼만 게인 K 는 예측을 얼마나 믿을지 결정

4.2 EKF 상태 벡터 x (16차원)

[0:3] 위치 $p = (p_x, p_y, p_z)$

단위: m, 세계 좌표계

[3:6] 속도 $v = (v_x, v_y, v_z)$

단위: m/s, 세계 좌표계

[6:10] 쿼터니언 $q = (q_w, q_x, q_y, q_z)$

$\|q\| = 1$, 세계 ← 몸체

[10:13] 가속도 바이어스 b_a

EKF가 실시간 추정

[13:16] 자이로 바이어스 b_g

EKF가 실시간 추정

4.3 공분산 행렬 P (16×16) — 상태 불확실도

P 는 상태 벡터 x 의 각 성분 간 공분산을 담은 행렬입니다.

$P[i, i]$ 가 크다 = 해당 상태 성분이 불확실하다 = Kalman 게인이 커진다 = 측정을 더 신뢰한다.

$$P = E[(x - \hat{x})(x - \hat{x})^T]$$

공분산

$$P_{pos} = 0.005, P_{vel} = 0.05, P_{att} = 10^{-4}$$

초기값: (단위: $m^2, m^2/s^2, rad^2$)

4.4 EKF 2단계 구조

PREDICT (예측)

이전 상태 + IMU 입력 → 새 상태 예측

$$x_{k+1|k} = f(x_k, u_k), \quad P = F P F^T + Q$$

측정
도입

UPDATE (업데이트)

카메라 측정으로 예측 오차 보정

$$x_{k|k} = x_{k+1|k} + K(z - Hx), \quad P = (I - KH)P(I - KH)^T + KRK^T$$

EKF와 KF의 차이: IMU 적분, 쿼터니언 갱신, 회전 변환 등은 모두 비선형 함수입니다.

EKF는 이 비선형 함수를 현재 상태 주변에서 1차 Taylor 전개(Jacobian)로 선형화하여 처리합니다.

Chapter 5 — IMU 예측 단계

5.1 운동 방정식 (Equations of Motion)

IMU 입력으로 이전 상태에서 다음 상태를 예측합니다. $dt = 1/200 \text{ s}$ (200 Hz 자이로 기준).

이산 운동 방정식

$$\begin{aligned}\mathbf{p}_{k+1} &= \mathbf{p}_k + \mathbf{v}_k dt + \frac{1}{2} \mathbf{a}_w dt^2 \\ \mathbf{v}_{k+1} &= \mathbf{v}_k + \mathbf{a}_w dt, \quad \mathbf{a}_w = \mathbf{R}(\mathbf{q}_k) (\tilde{\mathbf{a}} - \mathbf{b}_a) + \mathbf{g} \\ \mathbf{q}_{k+1} &= \mathbf{q}_k \otimes \exp(\text{tfrc12 } \boldsymbol{\omega}_{body} dt)\end{aligned}$$

쿼터니언 적분 상세: 각속도 벡터 $\boldsymbol{\omega}$ 의 크기가 $\text{angle} = \|\boldsymbol{\omega}\| \times dt$ 일 때:

쿼터니언 적분
(각도 크면)

$$d\mathbf{q} = [\cos \frac{\text{angle}}{2}, \frac{\boldsymbol{\omega}}{\|\boldsymbol{\omega}\|} \sin \frac{\text{angle}}{2}]$$

$$d\mathbf{q} \approx [1, \frac{1}{2} \boldsymbol{\omega} dt]$$

(각도 매우 작으면, 1차 근사)

5.2 선형화 — Jacobian F 행렬 (16×16)

EKF 예측 시 공분산 전파를 위해 운동 방정식의 Jacobian $\mathbf{F} = \partial \mathbf{f} / \partial \mathbf{x}$ 가 필요합니다.

대부분의 F는 단위 행렬이고, 다음 블록만 비단위 값을 가집니다:

$\mathbf{F}[\mathbf{p}, \mathbf{v}] = \mathbf{I} \times dt$	위치는 속도 $\times dt$ 만큼 변함
$\mathbf{F}[\mathbf{p}, \mathbf{b}_a] = -\frac{1}{2}\mathbf{R} \times dt^2$	바이어스 오차가 위치에 미치는 영향
$\mathbf{F}[\mathbf{p}, \mathbf{q}] = \frac{1}{2} \partial(\mathbf{R}_a) / \partial \mathbf{q} \times dt^2$	자세 오차가 가속도 방향 변환에 미치는 영향
$\mathbf{F}[\mathbf{v}, \mathbf{q}] = \partial(\mathbf{R}_a) / \partial \mathbf{q} \times dt$	자세 오차가 속도 변화에 미치는 영향
$\mathbf{F}[\mathbf{v}, \mathbf{b}_a] = -\mathbf{R} \times dt$	바이어스 오차가 속도에 미치는 영향
$\mathbf{F}[\mathbf{q}, \mathbf{b}_g] = \partial \mathbf{q}_{new} / \partial \mathbf{b}_g \times dt$	자이로 바이어스 오차가 자세에 미치는 영향

5.3 프로세스 노이즈 Q — 모델 불확실도

Q는 IMU 노이즈가 상태에 미치는 불확실도를 표현합니다. σ_a, σ_g 가 클수록 Q가 커지고

P가 빠르게 성장하여 다음 측정에서의 Kalman 게인이 커집니다.

Q 구성 및 공분산 전파

$$\mathbf{Q}[\mathbf{v}, \mathbf{v}] = \sigma_a^2 \cdot dt \cdot \mathbf{I}_3$$

$$\mathbf{Q}[\mathbf{p}, \mathbf{p}] = \frac{1}{4} \sigma_a^2 \cdot dt^3 \cdot \mathbf{I}_3$$

$$\mathbf{Q}[\mathbf{q}, \mathbf{q}] = \frac{1}{4} \sigma_g^2 \cdot dt \cdot \mathbf{I}_4$$

$$\mathbf{P} \leftarrow \mathbf{F} \mathbf{P} \mathbf{F}^\top + \mathbf{Q}$$

Pseudocode — EKF Predict

```
# IMU 예측 스텝 의사코드
def predict(x, P, accel_raw, gyro_raw, dt):
    p, v, q, ba, bg = unpack(x)
    R = quat_to_rot(q)
    a_body = accel_raw - ba
    w_body = gyro_raw - bg
    a_world = R @ a_body + [0, 0, -9.81]

    p_new = p + v*dt + 0.5*a_world*dt**2
    v_new = v + a_world*dt
    dq = angle_to_quat(w_body * dt)
    q_new = normalize(quat_mult(q, dq))

    F = build_jacobian(a_body, q, dt)
    Q = build_process_noise(sigma_a, sigma_g, dt)
    P_new = F @ P @ F.T + Q
    return pack(p_new, v_new, q_new, ba, bg), P_new
```

바이어스 보정
세계 좌표 가속도
위치 적분
속도 적분
쿼터니언 적분
선형화
공분산 전파

Chapter 6 — 중력 정렬 초기화

6.1 왜 초기화가 필요한가?

EKF의 초기 쿼터니언은 $\mathbf{q} = [1, 0, 0, 0]$ (항등 회전)으로 설정되어 있습니다.

이는 몸체 좌표계 = 세계 좌표계로 가정하는 것입니다. 하지만 실제 디바이스가 기울어져 있다면, IMU로부터 변환한 중력 방향이 틀려서 EKF가 즉시 발산합니다.

6.2 정지 상태에서의 가속도 = 중력 벡터

가속도계 모델에서 디바이스가 정지하면 $\mathbf{a}_{\text{world}} = 0$ 이므로:

$$\tilde{\mathbf{a}} \approx \mathbf{R}^T (0 - \mathbf{g}) + \mathbf{b}_a + \mathbf{n}_a$$

정지 가속도 = 중력

$$\approx -\mathbf{R}^T \mathbf{g}$$

(바이어스 · 노이즈 무시)

즉, 정지 시 가속도계 측정값은 중력 벡터가 몸체 좌표로 표현된 것

따라서 1초간의 정지 가속도 평균 $\mathbf{g}_{\text{body}} = \text{mean}(\tilde{\mathbf{a}})$ 는 몸체 좌표계의 중력 방향입니다.

세계 좌표계에서 중력은 $\mathbf{z}_{\text{world}} = [0, 0, 1]$ ($-\mathbf{g}$ 방향 기준)로 알려져 있으므로,

두 벡터를 정렬하는 회전을 초기 쿼터니언으로 사용합니다.

6.3 축-각도 방법으로 초기 쿼터니언 계산

$$\hat{\mathbf{g}}_{\text{body}} = \frac{\mathbf{g}_{\text{body}}}{\|\mathbf{g}_{\text{body}}\|}$$

중력 정렬 쿼터니언

$$\mathbf{axis} = \hat{\mathbf{g}}_{\text{body}} \times \mathbf{z}_{\text{world}}, \quad \theta = \arctan 2(\|\mathbf{axis}\|, \hat{\mathbf{g}} \cdot \mathbf{z})$$

$$\mathbf{q}_{\text{init}} = [\cos \frac{\theta}{2}, \mathbf{axis} \sin \frac{\theta}{2}]$$

Pseudocode — Gravity Alignment

중력 정렬 초기화 의사코드

```
def initialize_from_accel(accel_samples, gyro_samples):
```

```
    # 1초간(~60 샘플) 정지 데이터 평균
```

```
    g_body = mean(accel_samples)
```

몸체 좌표 중력 벡터

```
    g_hat = g_body / norm(g_body)
```

단위 벡터

```
    z_world = [0, 0, 1]
```

세계 좌표 중력 방향 (위쪽)

```
    # 두 벡터 사이의 회전 = 초기 자세
```

```
    axis = cross(g_hat, z_world)
```

회전축

```
    sin_a = norm(axis)
```

```
    if sin_a < 1e-6:
```

이미 정렬됨 or 완전 반대

```
        q = [1, 0, 0, 0] if g_hat[2] > 0 else [0, 1, 0, 0]
```

```
    else:
```

```
        axis /= sin_a
```

```
        angle = atan2(sin_a, dot(g_hat, z_world))
```

```
        q = [cos(angle/2), axis * sin(angle/2)]
```

```
    x[6:10] = normalize(q)
```

EKF 초기 쿼터니언 설정

```
    x[13:16] = mean(gyro_samples)
```

자이로 바이어스 초기 추정

```
    imu_buffer.reset()
```

정렬 전 IMU 샘플 버림

핵심: 중력 정렬 없이 EKF를 시작하면 $\mathbf{a}_{\text{world}} = \mathbf{R} @ \mathbf{a}_{\text{body}} + \mathbf{g}$ 에서 $\mathbf{R}$ 이 잘못되어

중력이 소거되지 않고 가속도 신호에 9.81 m/s^2 오프셋이 남아 위치가 수백 cm/s^2 발산합니다.

Chapter 7 — 피시아이 카메라 모델

7.1 핀홀 카메라 모델 (표준 카메라)

일반 카메라는 3D 점을 2D 이미지로 투영할 때 원근 투영(Perspective Projection)을 사용합니다:

$$u = f_x \cdot \frac{x}{z} + c_x, \quad v = f_y \cdot \frac{y}{z} + c_y$$

$$\mathbf{K} = \text{diag}(f_x, f_y, 1) \text{ with principal point } (c_x, c_y)$$

핀홀 투영

f_x, f_y : 초점거리(픽셀 단위), c_x, c_y : 주점(이미지 중심). T265: $f \approx 287$ px.

7.2 피시아이 카메라 — Kannala-Brandt 모델

T265는 약 160° FOV의 피시아이 렌즈를 사용합니다. 광각에서 핀홀 모델은 맞지 않으며

Kannala-Brandt 4계수 모델을 사용합니다:

$$\theta = \arctan\left(\frac{\sqrt{x^2 + y^2}}{z}\right)$$

$$r_d = \theta + k_1\theta^3 + k_2\theta^5 + k_3\theta^7 + k_4\theta^9$$

$$u = f_x \frac{x}{\sqrt{x^2 + y^2}} r_d + c_x$$

Kannala-Brandt
(입사각)

(방사 왜곡)

핀홀은 $\tan(\theta)$ 를 사용하지만 KB는 θ + 다항식 보정을 사용하여 광각에서도 정확합니다.

7.3 왜곡 보정 (Undistortion)

피시아이 이미지 → 핀홀 이미지로 변환합니다. 변환 후 이미지는 왜곡 없이 표준

카메라 모델이 적용됩니다. 출력 카메라 행렬 $\mathbf{K}_{\text{new}}$ 를 어떻게 설정하느냐가 중요합니다:

$\mathbf{K}_{\text{new}} = \mathbf{K}_1$ (원래 초점거리 유지)

원래 카메라와 같은 FOV / 크기

[*] 사용 (자연스러운 모습)

$\mathbf{K}_{\text{new}} = \text{estimateNewCameraMatrix}$
(balance=0.3)

$f_{\text{new}} \approx 112$ px (원래의 39%)

[*] 2.5× 줌아웃 — 이미지가 "퍼짐"

Pseudocode — Fisheye Undistortion

피시아이 → 핀홀 왜곡 보정

def undistort(img, K, D, K_new):

cv2.fisheye.undistortImage:

K = 원래 내부 파라미터 (Kannala-Brandt)

D = 왜곡 계수 [k1, k2, k3, k4]

K_new = 출력 핀홀 카메라 행렬

→ 각 출력 픽셀 (u,v)에 대해 역 KB 모델로 입력 픽셀 좌표를

계산하여 bilinear interpolation 으로 샘플링

h, w = img.shape[:2]

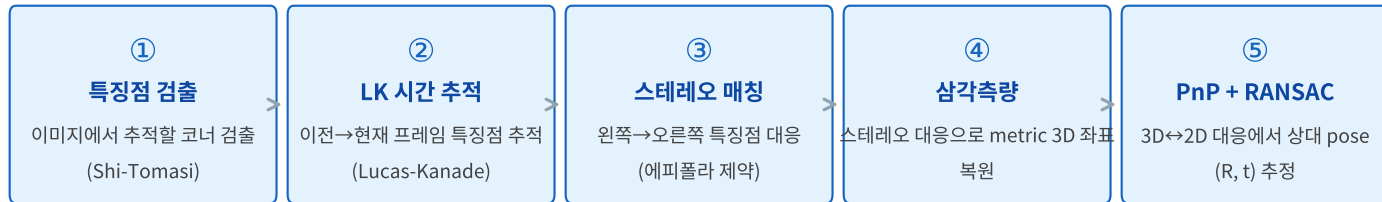
return cv2.fisheye.undistortImage(img, K, D, None, K_new, (w, h))

핵심 버그 사례: balance=0.3으로 자동 계산된 $\mathbf{K}_{\text{new}}$ 를 사용하면 $f_{\text{new}} \ll f_{\text{original}}$ 이 되어

이미지가 심하게 "퍼져" 보입니다. $\mathbf{K}_{\text{new}} = \mathbf{K}_1.\text{copy}()$ 로 원래 초점거리를 유지해야 합니다.

Chapter 8 — 스테레오 비주얼 오도메트리 (1/2)

8.1 전체 파이프라인



8.2 특징점 검출 — Shi-Tomasi Corner

이미지에서 추적하기 좋은 코너(Corner)를 최대 N개 검출합니다.

코너는 x, y 방향 모두로 명확한 기울기가 있어서 LK 추적이 안정적입니다.

$$M = \sum_{patch} \begin{pmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{pmatrix}_{2 \times 2}$$

$$\min(\lambda_1, \lambda_2)$$

Shi-Tomasi
(구조 텐서)

점수 = (두 고유값 중 작은 것이 크면 코너)

8.3 Lucas-Kanade 광학 흐름 (Optical Flow)

이전 프레임의 특징점 위치에서 현재 프레임의 대응 위치를 반복적으로 추정합니다.

가정: 작은 패치 내의 픽셀들은 같은 속도로 움직인다 (Aperture 가정).

$$I(x, y, t) = I(x + u, y + v, t + dt)$$

Lucas-Kanade
(밝기 보존 가정)

$$I_x \cdot u + I_y \cdot v = -I_t$$

(광학 흐름 방정식)

피라미드 LK: 큰 해상도 → 작은 해상도 순으로 반복 수렴 (큰 이동 처리)

Pseudocode — Lucas-Kanade Tracking

```
# LK 시간 추적 (이전→현재 프레임)
curr_pts, status, _ = cv2.calcOpticalFlowPyrLK(
    prev_gray,                # 이전 프레임
    curr_gray,                # 현재 프레임
    prev_pts,                 # 이전 특징점 위치 (N, 1, 2)
    None,
    winSize=(21, 21),        # 패치 크기
    maxLevel=3,              # 피라미드 레벨
    criteria=(TERM_CRITERIA_EPS | TERM_CRITERIA_COUNT, 30, 0.01)
)
ok = (status.ravel() == 1)    # 성공한 추적만 선택
tracked_pts = curr_pts[ok]
```

Chapter 8 — 스테레오 비주얼 오도메트리 (2/2)

8.4 스테레오 삼각측량 (Triangulation)

같은 3D 점 P가 왼쪽 카메라(u_l)와 오른쪽 카메라(u_r)에 각각 투영됩니다.

두 카메라 레이의 교점이 3D 점입니다. 스테레오 베이스라인 $b = 6.4$ cm에서

시차(Disparity) $d = u_l - u_r$ 로 깊이를 복원합니다:

$$Z = \frac{f \cdot b}{d} \quad (\text{깊이} = \text{초점거리} \times \text{스테레오 깊이} / \text{시차})$$

$$\sigma_Z \approx \frac{Z^2}{f \cdot b} \quad (\text{깊이 오차} \propto \text{깊이}^2)$$

$$\sigma_Z \approx \frac{4}{287 \times 0.064} \approx 22 \quad \text{T265에서 } Z=2\text{m: cm/pixel}$$

따라서 깊이 필터 $z_{\min}=0.15\text{m}$, $z_{\max}=2.0\text{m}$ 로 믿을 수 있는 범위만 사용합니다.

에피플라 제약: T265 카메라가 거의 평행이므로 세로 시차 $|\Delta y| < 3$ px 조건으로 아웃라이어 제거.

8.5 PnP — 3D-2D 대응에서 Pose 추정

이전 프레임에서 삼각측량한 3D 점들이 현재 프레임의 어느 픽셀에 투영되는지 알 때,

그 대응을 이용해 카메라 pose (R, t)를 추정합니다:

$$\{(\mathbf{P}_i^{3D}, \mathbf{p}_i^{2D})\}_{i=1}^N \quad \text{PnP 문제 주어진: (3D 점, 2D 투영)}$$

$$(\mathbf{R}, \mathbf{t}) \sum_i |\mathbf{p}_i^{2D} - \pi(\mathbf{R}\mathbf{P}_i + \mathbf{t})|^2 \quad \text{구하기: 최소화}$$

>> RANSAC — 아웃라이어에 강한 PnP

RANSAC은 무작위 소집합(Minimal Sample)으로 모델을 추정하고, 이 모델을 지지하는

점 개수(Inlier)가 가장 많은 모델을 선택합니다. 잘못 추적된 특징점(아웃라이어)에

강인합니다.

Pseudocode — PnP + RANSAC

PnP + RANSAC

def pnp(pts3d, pts2d, K):

```
    ok, rvec, tvec, inliers = cv2.solvePnPRansac(
        pts3d, pts2d, K, None,
        iterationsCount=100,
        reprojectionError=2.0, confidence=0.999,
        flags=SOLVEPNP_ITERATIVE)
```

```
    if not ok or len(inliers) < 12: return None
```

```
    R, _ = cv2.Rodrigues(rvec)
```

```
    t = tvec.ravel()
```

```
    if norm(t) > 0.30: return None
```

30cm 초과 → 아웃라이어

```
    return R, t, inliers
```

8.6 좌표계 변환 (핵심 주의사항)

PnP는 pts3d가 모두 같은 좌표계(이전 카메라 프레임)에 있어야 올바른 결과를 냅니다.

PnP 결과: $P_{\text{curr}} = R @ P_{\text{prev}} + t$ 로 변환 후 저장 → 새 스테레오 점과 같은 좌표계.

Chapter 9 — EKF 측정 업데이트와 게이팅

9.1 속도 측정 모델

PnP로부터 얻은 상대 이동 $\mathbf{t}_{metric}$ (이전 카메라 좌표계)을 세계 좌표 속도로 변환합니다:

$$\Delta \mathbf{p}_W = \mathbf{R}_{world} \mathbf{t}_{metric}$$

속도 측정 모델
(세계 좌표 변위)

$$\mathbf{v}_{vis} = \Delta \mathbf{p}_W / dt$$

(시각 속도 추정치)

$$\mathbf{z} = \mathbf{v}_{vis} = \mathbf{H} \mathbf{x} + \mathbf{n}$$

측정 방정식:

측정 행렬 $\mathbf{H}$ (3×16): $\mathbf{H}[0:3, 3:6] = \mathbf{I}_3$ (속도 성분만 관측).

측정 노이즈: $\mathbf{R}_{vel} = \text{diag}([0.3^2, 0.3^2, 0.3^2]) \text{ m}^2/\text{s}^2$ (시각 속도 추정 오차 $\sim 0.3 \text{ m/s}$).

9.2 칼만 게인과 업데이트

$$\mathbf{S} = \mathbf{H} \mathbf{P} \mathbf{H}^\top + \mathbf{R}_{vel}$$

EKF 업데이트 공식
(혁신 공분산)

$$\mathbf{K} = \mathbf{P} \mathbf{H}^\top \mathbf{S}^{-1}$$

(칼만 게인)

$$\mathbf{y} = \mathbf{v}_{vis} - \hat{\mathbf{v}}$$

(혁신 = 측정 - 예측)

$$\mathbf{x} \leftarrow \mathbf{x} + \mathbf{K} \mathbf{y}$$

(상태 업데이트)

$$\mathbf{P} \leftarrow (\mathbf{I} - \mathbf{K} \mathbf{H}) \mathbf{P} (\mathbf{I} - \mathbf{K} \mathbf{H})^\top + \mathbf{K} \mathbf{R}_{vel} \mathbf{K}^\top$$

(Joseph 형식)

9.3 Mahalanobis 거리 게이팅 — 점프 방지

PnP가 가끔 잘못된 결과를 낼 때(예: 큰 backward velocity), 이 값을 그대로

EKF에 적용하면 position이 갑자기 이전 위치로 점프합니다. 통계적 게이팅으로 이를 차단합니다:

$$d^2 = \mathbf{y}^\top \mathbf{S}^{-1} \mathbf{y}$$

게이팅
(Mahalanobis 거리 제곱)

$$d^2 > \chi^2_{3, 99.9\%} = 16.27 \Rightarrow$$

업데이트 거부 (아웃라이어)

$$d^2 \leq 16.27 \Rightarrow$$

업데이트 수용

$\chi^2(3, 99.9\%) = 16.27$: 3자유도(속도 x,y,z) 측정에서 통계적으로 가능한 범위 99.9% 내.

게이팅은 S의 크기(현재 불확실도)를 고려하므로, P가 작을 때 같은 혁신이 더 강하게 차단됩니다.

Pseudocode — EKF Visual Update with Gating

EKF 시각 업데이트 + Mahalanobis 게이팅

```
def update_visual(x, P, v_vis, R_vel, gate=16.0):
```

```
    H = zeros((3, 16)); H[0:3, 3:6] = eye(3)
```

```
    y = v_vis - x[3:6]
```

```
    S = H @ P @ H.T + R_vel
```

속도 관측

혁신

혁신 공분산

```
    # Mahalanobis 거리 게이팅
```

```
    md2 = y @ solve(S, y)
```

```
    if md2 > gate:
```

```
        return x, P, False
```

아웃라이어 → 거부

```
    K = P @ H.T @ inv(S)
```

칼만 게인

```
    dx = K @ y
```

```
    x_new = x + dx
```

```
    x_new[6:10] = normalize(x_new[6:10])
```

쿼터니언 재정규화

```
    I_KH = eye(16) - K @ H
```

```
    P_new = I_KH @ P @ I_KH.T + K @ R_vel @ K.T
```

Joseph 형식

```
    return x_new, P_new, True
```

Chapter 10 — ZUPT (Zero Velocity Update)

10.1 이중 적분 드리프트 문제

IMU 가속도를 이중 적분하면 위치를 얻지만, 바이어스와 노이즈가 누적되어 발산합니다. 카메라 업데이트가 없는 구간(폐쇄, 조명 불량 등)에서 EKF는 IMU 예측만 수행하게 되어 속도 오차가 쌓입니다. 디바이스가 실제로 정지해 있다면 이를 이용해 강제 보정합니다.

10.2 ZUPT 원리

정지 조건 감지: IMU 측정값에서 디바이스가 정지했음을 판단합니다.

감지 시: 속도 = 0 이라는 측정값(z = 0)을 EKF에 주입합니다.

효과: 속도 오차 누적을 주기적으로 초기화하여 위치 드리프트 억제.

$$||\tilde{\mathbf{a}}| - 9.81| < 0.8 \quad |\tilde{\omega}| < 0.08$$

정지 조건: m/s^2 AND rad/s

$$\mathbf{z}_{\text{zupt}} = \mathbf{0} = \mathbf{H} \mathbf{x} + \mathbf{n}_{\text{zupt}}$$

ZUPT 측정:

$$\mathbf{R}_{\text{zupt}} = \sigma_{\text{zupt}}^2 \mathbf{I}_3, \quad \sigma_{\text{zupt}} = 0.01 \text{ m/s (tight)}$$

$|\tilde{\mathbf{a}}| \approx 9.81$: 가속도 크기가 중력과 같다 = 정지 (추가 운동 가속도 없음).

$|\mathbf{w}_{\text{raw}}| \approx 0$: 회전이 없다. 두 조건 모두 만족 시 ZUPT를 적용합니다.

10.3 ZUPT EKF 업데이트

$$\mathbf{S} = \mathbf{H} \mathbf{P} \mathbf{H}^T + \mathbf{R}_{\text{zupt}}$$

ZUPT 업데이트

$$\mathbf{K} = \mathbf{P} \mathbf{H}^T \mathbf{S}^{-1}$$
$$\mathbf{y} = \mathbf{0} - \hat{\mathbf{v}} = -\hat{\mathbf{v}}$$

(혁신: 속도를 0으로)

$$\mathbf{x} \leftarrow \mathbf{x} + \mathbf{K} \mathbf{y}, \quad \mathbf{P} \leftarrow (\mathbf{I} - \mathbf{K} \mathbf{H}) \mathbf{P} (\mathbf{I} - \mathbf{K} \mathbf{H})^T + \mathbf{K} \mathbf{R}_{\text{zupt}} \mathbf{K}^T$$

Pseudocode — ZUPT

```
# ZUPT 의사코드
def zupt_if_stationary(bundle, ekf):
    a_mag = norm(mean([a for _, a in bundle.accel_samples]))
    g_mag = norm(mean([g for _, g in bundle.gyro_samples]))

    if abs(a_mag - 9.81) < 0.8 and g_mag < 0.08:
        # 정지 확인 → 속도 = 0 업데이트
        H = zeros((3, 16)); H[0:3, 3:6] = eye(3)
        R_zupt = eye(3) * (0.01 ** 2) # 매우 타이트한 노이즈
        y = -ekf.x[3:6] # 혁신 = 0 - v_current
        S = H @ ekf.P @ H.T + R_zupt
        K = ekf.P @ H.T @ inv(S)
        ekf.x += K @ y
        ekf.x[6:10] = normalize(ekf.x[6:10])
        I_KH = eye(16) - K @ H
        ekf.P = I_KH @ ekf.P @ I_KH.T + K @ R_zupt @ K.T
```

순서 중요: ZUPT는 시각 업데이트 이후 마지막에 수행합니다.

시각 업데이트 전에 ZUPT를 하면 카메라 업데이트가 ZUPT를 덮어써 효과가 없어집니다.

정지 감지 임계값 (0.8 m/s^2 , 0.08 rad/s)은 일반 보행 중 ZUPT가 오발동하지 않도록 설정.

Chapter 11 — 전체 시스템 파이프라인

11.1 매 프레임 처리 흐름



11.2 파라미터 정리

$\sigma_a = 0.01 \text{ m/s}^2/\sqrt{\text{Hz}}$	가속도 노이즈 — T265 스펙 기준
$\sigma_g = 0.002 \text{ rad/s}/\sqrt{\text{Hz}}$	자이로 노이즈
$r_{\text{vel}} = 0.3 \text{ m/s}$	EKF 시각 속도 측정 노이즈 (std)
$\chi^2_{\text{gate}} = 16.0$	Mahalanobis 게이팅 (99.9%)
$\text{max_corners} = 250$	특징점 최대 개수 (LK/PnP 성능 상한)
$z_{\text{min/max}} = 0.15\sim 2.0 \text{ m}$	유효 삼각측량 깊이 범위

- 핵심 설계 원칙:
- ① IMU는 예측, 카메라는 보정 — 두 센서를 EKF로 융합
 - ② 모든 3D 점은 같은 좌표계(이전 카메라 프레임)에 있어야 PnP가 올바름
 - ③ Mahalanobis 게이팅으로 잘못된 시각 업데이트 차단
 - ④ ZUPT로 정지 시 이중 적분 드리프트 억제 (시각 업데이트 다음에 실행)